Meta-modeling and instance generation with Refinery

Niels Van den Broeck, Robbe Stuer

1 Introduction

In this assignment, a meta-model is made in Refinery that specifies the structure and rules of a role-playing game. From this meta-model, instances (or models) can be generated specifying different roles, skills, weapons, and so on.

2 Model Setup and Design Choices

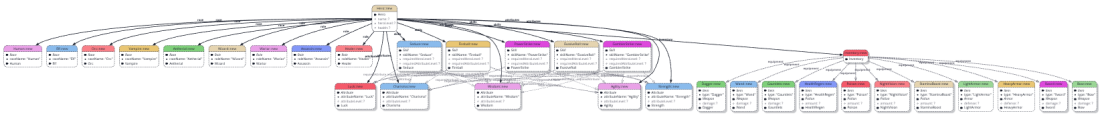


Figure 1: Complete meta-model.

Before jumping into our design choices, there is a structure that we had to follow.

2.1 Specification

First, we made the main character of the model, which is the hero with a name and level. Using polymorphism, we can create races and roles to which the hero is assigned. Additionally, attributes, skills, and inventory are added to the hero's class fields, each having their own representation and functionality. The inventory includes items. These can be armor, protecting the hero, 1 or 2 weapons, helping him in battle, and potions that may be useful in certain scenarios.

Listing 1: Hero class definition

```
class Hero {
    string name;
    int heroLevel;
    int health;
    contains Race race;
    contains Role role;
    contains Attribute[1..*] attributes;
    contains Skill[] skills;
    contains Inventory inventory;
}
```

2.2 Theme

As the assignment is fairly open and we can choose to make our game unique in any way we want, we implemented some extra features. To begin with, we decided to come up with our own theme for the game. The game includes different races like Vampires and Aetherials, roles like Assassins and Healers, attributes including Luck and Charisma, skills such as Fireball and EvasiveRoll, weapons like Gauntlets and Wands, and potions such as NightVision and StaminaBoost.

2.3 Preventing External Object Spawning

We decided to make this model purely character-based. It would be possible to have objects outside the hero like other characters (enemies, allies) and items like collectibles laying around in the area. We considered including this, but agreed not to since this would introduce unnecessary complexity and distract from the main focus of the assignment. To prevent objects from spawning outside the hero object, we made sure to add the "contains" keyword when declaring them in the Hero class. In this way, they can occur only in this class.

3 Constraints

Since there are all kinds of constraints, and the possibilities are endless, we focused on the most important and interesting ones, including a few additional examples to demonstrate what is possible.

3.1 scopes

In order to have exactly 1 Hero, 1 Race, 1 Role and 1 Armor, we can simply add scopes like in listing 2, specifying exactly 1 object have to spawn. Originally, Weapon was restricted in the same way, but was later moved to listing 6 for an exception explained further in this report.

Listing 2: scope constraints

```
scope Hero = 1.
scope Race = 1.
scope Role = 1.
scope Armor = 1.
```

3.2 Out of bounds

To ensure that the hero level, health or other values do not get below 0 or above a specified limit, we added constraints like in listing 3. this rule triggers if for a hero h, its level is below or equal to 0 or above 100. Such rules are added for each variable like health, attributelevel, and so on.

Listing 3: Hero level out of bounds

```
error HeroLevelOOB(Hero h) <->
    heroLevel(h) <= 0 || heroLevel(h) > 100.
```

3.3 links

An important part of our model is that skills can only be learned when one or more certain attributes have reached a certain level. The first rule ensures that an attribute that is required for a skill, is higher than the actual level of the attribute. The second rule is specifically for a given skill, ensuring the attribute is present when this skill is learned. We added more of these rules for each skill in the model file.

Listing 4: Skill requires attribute level

```
error skillRequiresAttributeLevel(Skill s, Attribute a) <->
    requiredAttribute(s, a),
    attributeLevel(a) < requiredAttributeLevel(s).

error PowerstrikeStrength(PowerStrike s) <->
    requiredAttribute(s,a),
    attributeName(a) != "Strength".
```

The rule in listing 5 shows that, for each pair of skills, they are checked to be equal. If so, it will trigger, preventing the hero from having duplicate skills. This is done for attributes as well.

Listing 5: Duplicate skills

```
error duplicateSkills(Skill s1, Skill s2) <->
    s1 != s2,
    skillName(s1) == skillName(s2).
```

An expansion we wanted to include was for the Assassin to be able to wield two daggers, while it was previously designed for a hero to only wield one weapon. We achieved this by creating two rules. The first rule makes sure that all other roles can only wield one weapon, while the second rule ensures that the Assassin is not able to wield two weapons other than the daggers. For example, a Healer with two daggers will fail in the first rule, and an Assassin with two bows will fail in the second rule.

Listing 6: Only Assassin double daggers

```
scope Weapon = 1..2.

error WeaponLimit(Role r) <->
    roleName(r) != "Assassin",
    count {Weapon(_i)} > 1.

error AssassinDaggers(Role r, Weapon w) <->
    roleName(r) =="Assassin",
    type(w) != "Dagger",
    count {Weapon(_i)} > 1.
```

4 Generated example model

Below, an example of a generated model is shown, illustrating the structure and functionality of our RPG. The hero is the central piece of our model, containing other useful related objects. Attributes are the only objects that may have multiple references to itself, one from the hero, and one or more from the skills that require this attribute. The inventory has multiple items located at the right of the generated model.

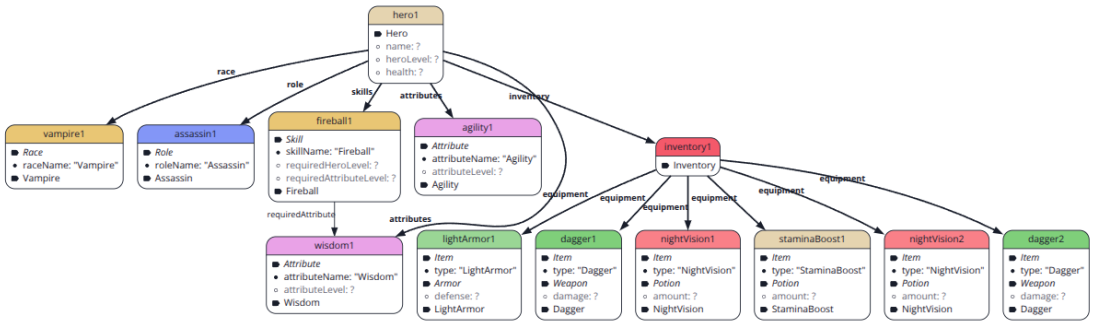


Figure 2: Generated example model.